

report

May 24, 2026



UNIVERSIDAD DE COLIMA

Reporte Técnico Del Sistema De Minería De Texto Y Embeddings

Minería de Datos

Universidad de Colima

Facultad de Ingeniería Mecánica y Eléctrica

6°B

Christian David Sánchez Sánchez

Coquimatlán, Colima, México.

24 de mayo de 2026

0.1 CONTENIDO

1. Introducción
2. Desarrollo de la Práctica
3. Normalización de Corpus
4. Tokens
5. One-Hot Encoding
6. Creación de los Pares (contextualización)
7. Calculo de TF-IDF
8. Skipgram
9. Búsqueda Semantica

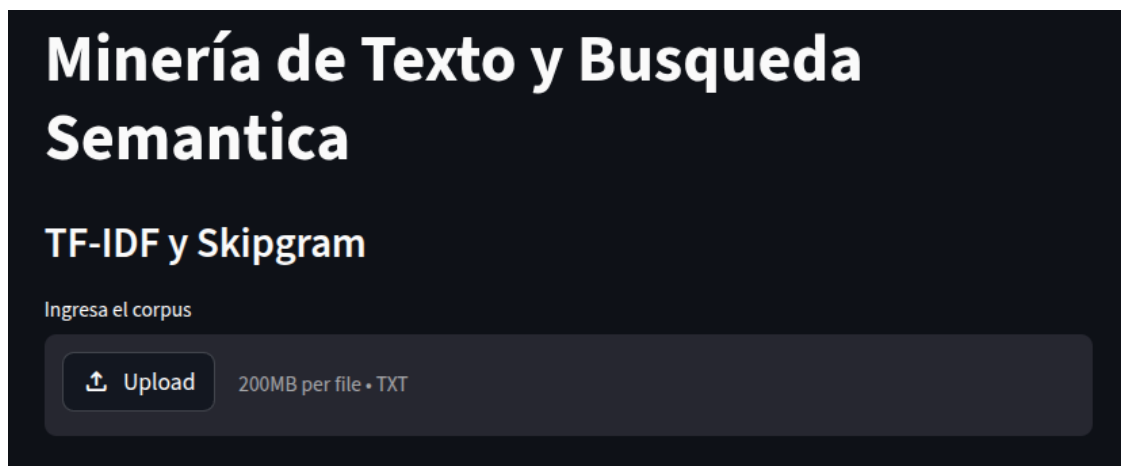
0.2 Introducción

El objetivo del desarrollo de este software es lograr el entendimiento de algoritmos de minería de texto y las bases de las redes neuronales mediante su implementación en un caso real. Mediante el uso de un corpus de más de 700 tokens únicos, se buscó lograr la implementación de los algoritmos TF-IDF, *One-Hot Encoding*, *Pairs* y *Skip Gram* para, con este entendimiento, lograr crear encoders didácticos pero funcionales. La metodología seguida para el desarrollo del software fue una implementación (no purista) de un patrón basado en *Programación Orientada a Objetos* y la *Inyección de Dependencias* para garantizar la legibilidad y el mantenimiento del código. El código fue escrito en *Python*, utilizando la librería *Streamlit* para la creación de una UI decente sin necesidad de tener problemas de UX.

Para consultar el código, ingresar al [repositorio en GitHub](#)

0.3 Desarrollo de la Práctica

Al ingresar al sistema, se nos pedirá subir un archivo .txt (nuestro corpus) para realizar las respectivas operaciones.



Una vez subido el corpus, el sistema comenzará a trabajar en el calculo de los siguientes elementos:

0.4 Normalización del Corpus

En Streamlit, todo archivo se sube como una lista de bytes por lo que tenemos que decodificarlo a un formato como UTF-8. Para esto, existen métodos sencillos de aplicar. Además, tenemos que eliminar los símbolos de puntuación para quedarnos solo con palabras y espacios válidos.

```
# Decodificar los bytes a un string en formato UTF-8
file = file.getvalue().decode("utf-8")
# Remover los simbolos de puntuación
file = self.__remove_punctuation(file)
```

0.5 Tokens

Una vez nuestro corpus está limpio, podemos comenzar con la minería de texto. Para esto, lo primero a obtener es la lista de stopwords basados en un lenguaje. Para esto, nos apoyamos de la

librería `nltk` la cual ofrece sets de stopwords en diferentes lenguajes.

```
from nltk.corpus import stopwords
stop_words = stopwords.words(language)
```

Despues (mediante la comprensión de listas), iteramos sobre cada palabra de nuestro corpus, desechando las que están vacías o en el set de stopwords y, por último, eliminamos las palabras repetidas.

```
tokens = np.array(
    [value for value in file.split() if value.lower() not in stop_words and value != ""]
)
tokens = pd.unique(tokens)
```



Vocabulario del corpus

Total de palabras: 652

value
INTELIGENCIA
ARTIFICIAL
IA
RAMA
INFORMÁTICA
BUSCA
DESARROLLAR
SISTEMAS
CAPACES
REALIZAR

0.6 One-Hot Encoding

Una vez tenemos nuestros tokens, crearemos una representación vectorial binaria de cada uno de ellos. Para esto, creamos una matrix de `len(tokens) x len(tokens)`, donde en cada intersección de las palabras existirá un 1 y en todo lo demás un 0 (creando una matriz identidad). Gracias a esta pequeña conversión, los algoritmos pueden procesar variables categóricas.

```
zeros = np.zeros((len(tokens), len(tokens)))
for i in range(len(tokens)):
    zeros[i][i] = 1
df = pd.DataFrame(data=zeros, columns=tokens)
```

One-hot encoding de los datos

	INTELIGENCIA	ARTIFICIAL	IA	RAMA	INFORMÁTICA	BUSCA	DESARROLLAR	SISTEMAS
INTELIGENCIA	1	0	0	0	0	0	0	0
ARTIFICIAL	0	1	0	0	0	0	0	0
IA	0	0	1	0	0	0	0	0
RAMA	0	0	0	1	0	0	0	0
INFORMÁTICA	0	0	0	0	1	0	0	0
BUSCA	0	0	0	0	0	1	0	0
DESARROLLAR	0	0	0	0	0	0	1	0
SISTEMAS	0	0	0	0	0	0	0	1
CAPACES	0	0	0	0	0	0	0	0
REALIZAR	0	0	0	0	0	0	0	0

Nota: Es importante que NO se ordenen los tokens de ninguna forma, ya que se perdería el contexto requerido.

0.7 Creación de los pares (contextualización)

Ahora, con los tokens, one-hot y el tamaño de la ventana, calcularemos los pares para cada uno de los tokens en sus posiciones (lista de tuplas). Para esto, lo primero que realizamos fue la iteración de cada uno de los tokens en su posición *i*.

```
results: list[tuple[int, int]] = list()
sumIds = 0
for i in range(0, len(tokens)):
```

La variable sumIds es un acumulador para saber cuantos elementos tengo hacia atras (con mi ventana de contexto como limite)

Una vez realizado esto, tenemos que obtener el nombre de las columnas de la ventana de contexto actual (mediante el uso de slices y máximos para no caer fuera de rango). Despues, recorreremos mediante indices los headers correspondientes a la context window válida (actual) y vamos almacenando sus indices, ignorando el que es igual al indice padre.

```
subdf_columns = tokens[max(i - slidingWindow, 0):i + slidingWindow + 1]
for j in range(len(subdf_columns)):
    if tokens[i] == subdf_columns[j]:
        continue
    results.append((i, max(i - slidingWindow, 0) + j))
```

Finalmente, retornamos los pares creados.

Pares creados con la ventana de contexto en 2

Total de pares: 2602

Palabra 1	Palabra 2
INTELIGENCIA	ARTIFICIAL
INTELIGENCIA	IA
ARTIFICIAL	INTELIGENCIA
ARTIFICIAL	IA
ARTIFICIAL	RAMA
IA	INTELIGENCIA
IA	ARTIFICIAL
IA	RAMA
IA	INFORMÁTICA
RAMA	ARTIFICIAL

0.8 Calculo del TF-IDF (Frecuencia de Terminos - Frecuencia Inversa de Documento)

Ahora, calcularemos el TF-IDF con los datos que hemos recabado.

Para calcular este índice, se necesitan dos elementos principales:

- *Documentos*: Al tener un solo corpus, trataremos cada párrafo como un solo documento debido a que nuestro corpus lo permite. Al existir corpus con párrafos de unas cuantas palabras, también fue agregada la opción de hacer la división por cantidad de renglones.
- *Tokens*: Los tokens que obtuvimos mediante el procesamiento del corpus.

El algoritmo propuesto es sencillo, constando de dos etapas principales las cuales se apoyan de un hashmap para mejorar el rendimiento.

En la *primera etapa*, se iteran todos los tokens y, en cada iteración, se crea una variable **total** la cual almacenará en cuantos documentos aparece este token en específico. Esta parte es esencial en el cálculo del IDF.

```
freq = dict()
```

```
# Fill the map with the amount of documents containing each token
for token in tokens:
    total = 0
    # For get the occurrences of each tokens in all documents
    for doc in docs:
```

```

if doc.count(token) > 0:
    total+=1

```

```

freq.update({token: total})

```

En la segunda etapa, se realizarán los calculos del TF e IDF para poder calcular TF-IDF. Las formulas a calcular son:

$$TF = \log\left(\frac{\text{frecuencia de aparición del token en el documento}}{\text{número total de terminos en el documento}}\right)$$

$$IDF = \frac{\text{total de documentos en el corpus}}{\text{número de documentos que contengan el token}}$$

$$TF\text{-}IDF = TF * IDF$$

Estos tres calculos son realizados en esta sección:

```

# Calculate the TF-IDF index for each document
scores = list()
for doc in docs:
    row = list()
    doc_tokens = self.get_tokens(doc, language=lang)
    for token in tokens:
        tf = doc.count(token) / len(doc_tokens)
        idf = np.log(len(docs) / freq.get(token, 0.00001))
        row.append(tf * idf)

    scores.append(row)

```

Como podemos ver, el código se apoya en el hashmap calculado previamente. Además, se agrega un 0.00001 para evitar divisiones entre 0.

RESULTADOS DEL TF-IDF

Total de documentos: 74

	INTELIGENCIA	ARTIFICIAL	IA	RAMA	INFORMÁTICA	BUSCA	DESARROLLAR	SISTEMAS	CAPA
0	0.1634	0.1634	0.1585	0.3242	0.4782	0.4012	0.4012	0.2224	0.4
1	0.1839	0	0.0594	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	
3	0	0	0.0594	0	0	0	0	0	
4	0.1634	0.1634	0.1585	0	0	0	0	0	
5	0	0	0.0679	0	0	0	0	0	
6	0.1634	0.1634	0.1585	0	0	0	0	0	
7	0	0	0	0	0	0	0	0	
8	0	0	0	0	0	0	0	0	
9	0.1471	0	0.0475	0	0	0	0	0	

0.9 Skipgram

Para la implementación de skipgram, se creó una red neuronal simple (la base de skipgram) la cual fue entrenada con 10,000 épocas, donde en cada época se itera sobre los pares y las ventanas de contexto establecidos previamente.

El flujo es:

1. Entrar a bucle de 10,000 épocas
2. Inicializar variable de pérdida total en 0
3. Iterar sobre todos los pares (contexto) de nuestro corpus
4. Obtener el vector de entrada `input_vector = input_weights[target_word]` basados en la palabra objetivo (la dueña de la ventana de contexto)
5. Realizamos el forward pass del embedding de entrada con la segunda y última capa, obteniendo las probabilidades.

```
output_vector = np.dot(input_vector, output_weights)
output_probs = self.__softmax(output_vector)
```

6. Calculamos los gradientes de los valores mediante gradient descent (descenso de gradiente).

```
input_grad = np.dot(output_weights, error)
output_grad = np.outer(input_vector, error)
```
7. Con base a los gradientes calculados y una tasa de aprendizaje (normalmente con un valor bajo para que converga poco a poco), actualizamos los pesos. Se actualiza el peso del input

especifico pero de todos los de la salida, ya que todos influyen al hacer el forward pass.

```
input_weights[target_word] -= learning_rate * input_grad
output_weights -= learning_rate * output_grad
```

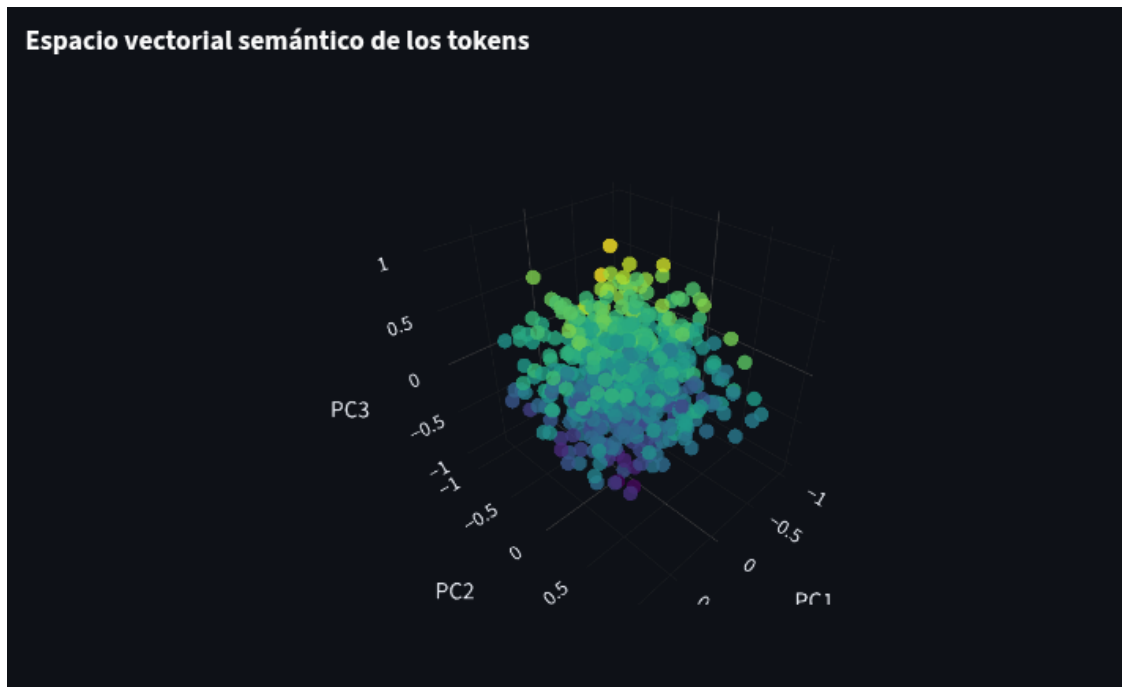
8. Calculamos el error basados en la palabra de contexto actual para ir acumulando los valores.

```
total_loss += -np.log(output_probs[context_word])
```

9. Volver a 2

Este flujo se repite hasta terminar las epocas, donde la perdida se abrá minimizado y los pesos de entrada serán los embeddings, donde a cada token le pertenece un embedding de 300 dimensiones.

Para la **visualización** del espacio vectorial, aplicamos un algoritmo de reducción de dimensionalidad (PCA) para convertir las 300 dimensiones en 3 sin perder información valiosa. De este modo, logramos modelar nuestro espacio vectorial.



Busqueda Semantica Para la busqueda semántica, se le muestra al usuario la lista de tokens presentes en el corpus dado y, una vez lo selecciona, se toma el embedding de esa palabra mediante su indice de los pesos de entrada.

Seguido de esto, tenemos que aplicar la formula de similitud de cosenos del embedding i contra todos los demás (a excepción de si mismo) para obtener los embeddings o tokens más similares.

```
for i, row in enumerate(input_weights):
    if i == token_index:
        continue

    data.append([tokens[i], self.__cosine_similarity(word_embedding, row)])
```

Esto nos genera una matriz de dos columnas (token, valor) pero esta contendrá absolutamente todos los valores, pero nosotros solo queremos obtener los que están dentro de la ventana de contexto del token en cuestion. Para esto, aplicamos los mismos máximos y mínimos que ya aplicamos anteriormente.

```
# Create a dataframe with the given data ordered
similarity = pd.DataFrame(columns=["Token", "Similarity"], data=data, index=None)
# Order in descending order
similarity.sort_values(by="Similarity", ascending=False, inplace=True)

# Based on the context window, we retrieve the N closest elements
return similarity[max(0, token_index - context_window):min(len(tokens) - 1, token_index + context_window)]
```

Podemos ver que primero ordenamos descendientemente para dejar los valores más altos primero y, despues, hacemos slices para que se evite el overflow de la ventana de contexto.